PDF QES Signer – User Manual

Installation, Configuration, and Signing Workflow

A GUI tool for visually placing and applying qualified electronic signatures (QES)

Contents

1 Introduction

PDF QES Signer is a desktop application for visually placing signature fields in PDF documents and applying **qualified electronic signatures (QES)** to them, using either a PKCS#11 hardware token (smartcard / USB token) or a PFX/PKCS#12 certificate file. It runs on Linux and Windows.

This manual covers everything that happens *after* installation: setting up a signing method, creating a certificate, signing a document, checking signatures, and understanding every field in the Settings dialog. For the legal background on what "qualified" means and how it differs from simple or advanced electronic signatures, see the companion document *Electronic Signatures Explained* (`docs/signatur-stufen.pdf`).

The screenshots in this manual are taken on Linux; the Windows interface is visually identical (both use the Qt framework).

---

2 Installation

Full installation instructions, including prerequisites, are in the project's `README.md`. In short:

Linux (one-liner):

```
bash <(curl -fsSL
↪  https://codeberg.org/pitbo/pdf-qes-signer/raw/branch/master/setup_pdf_signer.sh)
```

Windows (PowerShell):

```
irm 'https://codeberg.org/pitbo/pdf-qes-signer/raw/branch/master/setup_pdf_signer.bat'
↪  -OutFile "$env:TEMP\setup_pdf_signer.bat"; & "$env:TEMP\setup_pdf_signer.bat"
```

Both installers ask for an **update channel** (`stable`, recommended, or `develop` for pre-releases) and create a launcher you can use afterwards:

Linux: `./start_signer.sh [PDF_FILE]` (or the `pdf-signer` command if installed via the one-liner)
Windows: `start_signer.bat [PDF_FILE]`

Once the application starts for the first time, no signing method is configured yet. The next three chapters walk through setting one up before you open your first document.

---

3 Setting Up a Signing Method

To produce a cryptographic signature at all, the application needs access to two things: a **private key**, known only to you, and the matching **certificate**, which anyone verifying the signature can use to identify you (see chapter 17 for what a certificate and its "chain of trust" actually are). This chapter configures where that key and certificate come from — before you can sign your first document, you need to complete it once.

The choice below is mainly about *where the private key lives*:

- A **hardware token** (smartcard/USB token) never lets the private key leave the device — it signs internally on the card. This is what eIDAS requires for a legally **qualified** signature, and is covered in chapter 5.
- A **PFX/P12 file** stores the (password-encrypted) private key directly on disk. This is more convenient but offers weaker protection, and in practice is normally paired with a self-signed certificate for internal or testing use rather than a qualified signature (chapter 4).

Open **Settings → Settings...** (`Ctrl+,`). The first page, **Token / Signature**, controls which certificate is used to sign (Figure 2 for PFX mode, Figure 3 for hardware-token mode).

Method — a dropdown with two choices, each with its own set of fields below it:

Value	Meaning
`Key & Certificate (P12/PFX)`	Sign using a certificate + private key stored in a `.p12`/`.pfx` file on disk.
`Hardware Token (PKCS#11)`	Sign using a smartcard or USB token via a PKCS#11 driver library.

3.1 Key & Certificate (P12/PFX) fields

- **Name:** read-only, shows the Common Name (CN) of the currently loaded certificate, filled in automatically once the file has been read successfully.
- **P12/PFX file:** path to the certificate file, with a Browse button.
- **Show Certificate** button: opens a read-only viewer with the certificate's subject, issuer, validity period, and fingerprints.
- **Generate Key...** button: opens the certificate generator described in chapter 4.
- A small grey hint line beneath the path shows whether the file is password-protected.

3.2 Hardware Token (PKCS#11) fields

- **Name:** identical to the P12/PFX setting above — read-only, shows the CN of the certificate currently read from the token.
- **Library path (.so / .dll):** the PKCS#11 driver library provided by your token's manufacturer (e.g. `libpkcs11tcos_SigG_PCSC.so` for Telesec TCOS cards — see chapter 5 for how to find this).
- **Key ID:** the `CKA_ID` of the private key on the token, shown as a hex string. Retrieve it with:

```
pkcs11-tool --module /path/to/your-library.so --list-slots
```

- **PIN (test only):** used only by the *Test Token* buttons below, never saved to disk. Leave the PIN field on the main window empty to use a hardware PIN pad instead of typing the PIN on the computer.
- **Test Token (no PIN)** / **Test Token (with PIN)** buttons: verify the library path and Key ID resolve to a valid certificate *before* you attempt to sign a real document.

3.3 Shared settings (both modes)

- **Complete certificate chain when signing (AIA):** if enabled, a missing root CA certificate is downloaded via Authority Information Access (AIA) and embedded in the signature. Recommended for long-term archival (PAdES-LT / ETSI EN 319 132). You are asked to confirm the first time (Figure 1); after that the certificate is cached locally (`~/.config/pdf-signer/aia_cert_cache/`) and embedded silently.
- **Default Document Restriction (docMDP):** the default choice offered in the docMDP dialog (chapter 10) the first time you sign a document — *No restriction*, *Form fields & further signatures allowed* (recommended), or *No further changes allowed*.

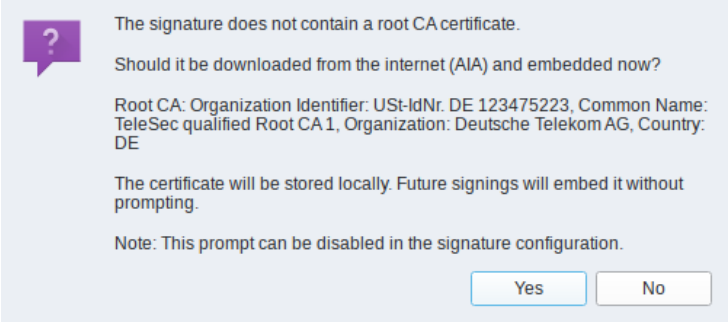

Figure 1. *The AIA confirmation dialog shown the first time a missing root CA certificate needs to be downloaded and embedded.*

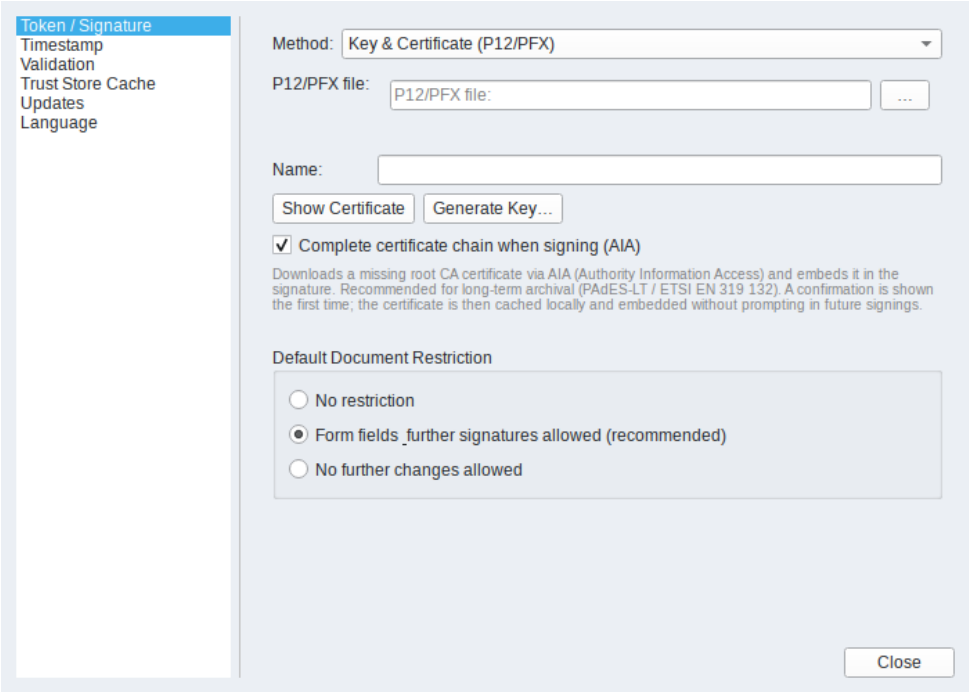

Figure 2. *Settings dialog, Token/Signature page in PFX mode.*

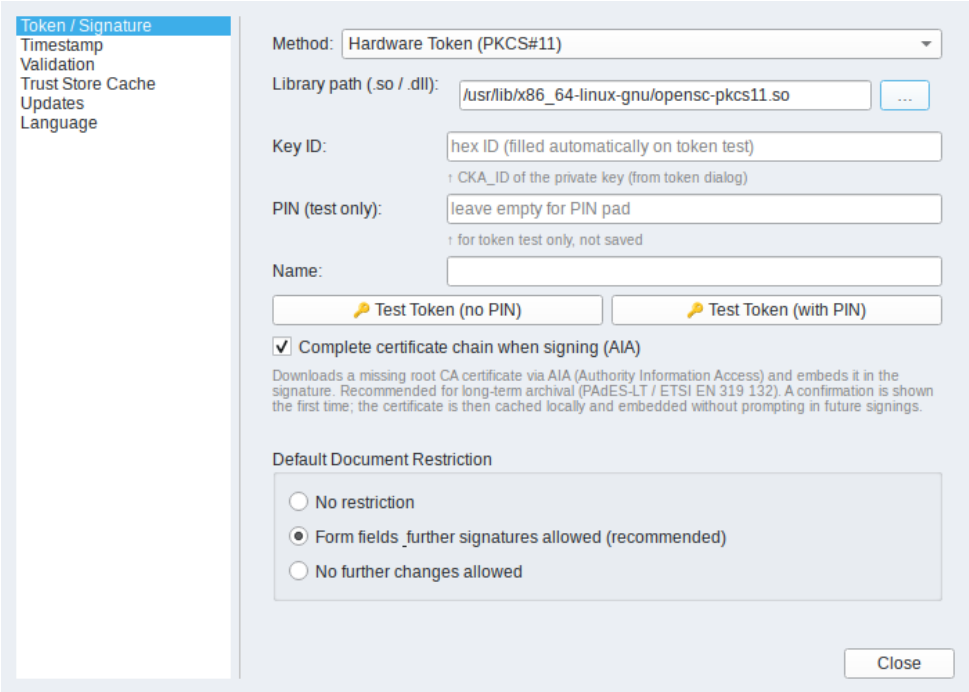

Figure 3. *Settings dialog, Token/Signature page in PKCS#11 mode.*

4 Creating Your Own Certificate

A certificate is what lets someone else verify who created a signature: it binds a name (and optionally organization, email) to a public key, and is normally only trustworthy because it was issued — digitally signed — by a Certificate Authority (CA) that verified that identity beforehand (see chapter 17). PDF QES Signer does not issue such CA-signed certificates itself. For a legally qualified signature you need one from a licensed Qualified Trust Service Provider (QTSP), or your hardware token may already come with one pre-installed (chapter 5).

This chapter's certificate generator instead creates a **self-signed** certificate, where you vouch for your own identity instead of a CA doing so. That does not make the resulting signature weak: cryptographically it still meets the eIDAS Article 26 requirements for an **Advanced Electronic Signature** — uniquely linked to you, created under your sole control, and tamper-evident (see chapter 17) — which is a clear step above a plain "I typed my name" electronic signature. What it is *not* is a **Qualified** Electronic Signature, since that additionally requires a qualified certificate from a licensed QTSP. In practice this only matters where the law specifically demands the written form (e.g. § 126 BGB) — for everything else, an Advanced Electronic Signature is legally sufficient and usually the more practical choice. See *Electronic Signatures Explained* (`docs/signatur-stufen.pdf`) for the full comparison of the three levels.

If you don't have a certificate yet, click **Generate Key...** on the Token/Signature settings page (PFX mode). This opens the certificate generator (Figure 4), which creates a private key and a **self-signed** X.509 certificate and saves both together as a password-protected `.p12`/`.pfx` file.

> **Important:** a self-signed certificate is not listed in any public trust store (the Mozilla CA bundle used for validation, or an EU Trust Service List). Signatures made with it will show as "unknown root" / untrusted to anyone else unless they manually import your certificate into their own trust store first. This does not change its legal standing as an Advanced Electronic Signature (see above) — it only means recipients must obtain your certificate out of band instead of relying on a publicly trusted chain. PAdES-LTA (long-term archival with embedded revocation status) is also not possible, since a self-signed certificate has no OCSP responder.

Key parameters:

- **Key type:** EC P-256, EC P-384, EC P-521 (default), RSA 3072, or RSA 4096. The signature hash algorithm is chosen automatically to match (e.g. SHA-512 for P-521).
- **Validity:** 1, 2, 3 (default), 5, or 10 years.
- **S/MIME encryption:** if checked, adds the key-usage flags needed to also use the certificate for encrypting emails, not just signing.

Certificate holder:

- **Common Name (CN):** your name, e.g. "Jane Doe".
- **Organization:** optional.
- **Country:** a two-letter ISO code (e.g. `DE`); the country name is shown live next to the field, and an invalid code is flagged in red.
- **Email:** optional; if set, it is embedded as a Subject Alternative Name.

File & password protection:

- **Save to:** destination path for the `.p12` (Linux/macOS) or `.pfx` (Windows) file.
- **Password / Confirm password:** protects the private key inside the file. This is what you later enter in the main window's PIN/Password field (chapter 6) when signing.

OpenSSL-equivalent command: a read-only box shows the exact `openssl` command that would produce the same result, with **Run** (executes it directly, without saving the intermediate PEM files to disk) and **Copy** buttons. This is provided for transparency and reproducibility — you can verify or reuse the exact command outside the application if you prefer.

Click **Generate** to create the file. On success, the new certificate is automatically selected as the active PFX file and its CN is shown on the settings page.

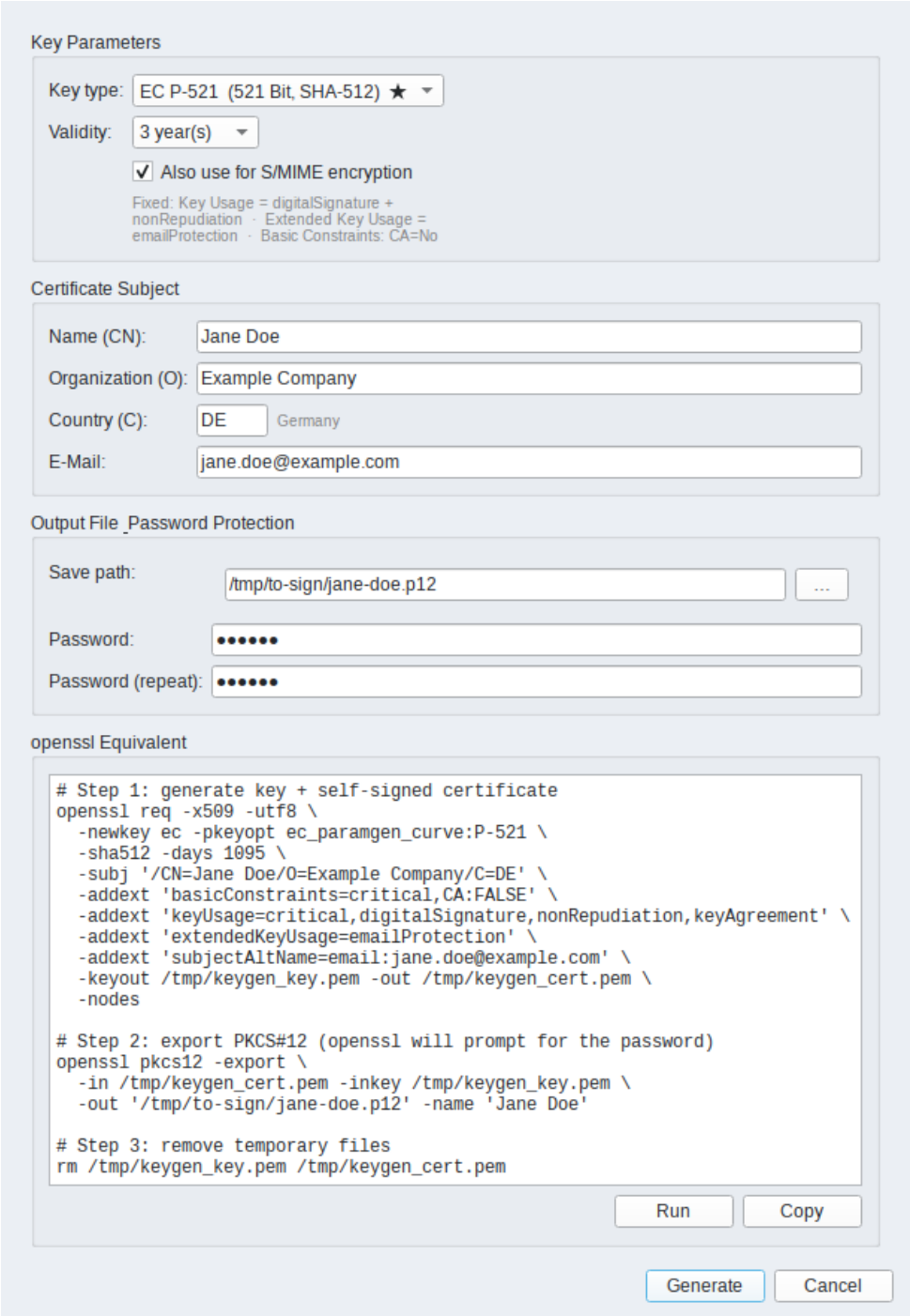

Figure 4. *Certificate generator dialog with the Subject and File sections filled in.*

---

5 Setting Up a PKCS#11 Hardware Token

For a hardware token (smartcard or USB token), switch **Method** to *Hardware Token (PKCS#11)* on the Token/Signature settings page.

1. **Find the driver library.** Your token manufacturer provides a PKCS#11 shared library (`.so` on Linux, `.dll` on Windows). This is *not* included with PDF QES Signer and must be installed separately. As one example: German Telesec **TCOS 3.0 SigG** cards require the proprietary `libpkcs11tcos_SigG_PCSC.so` from Deutsche Telekom Security — these cards are not properly supported by the open-source OpenSC middleware.

2. **Verify the card is recognized** before configuring the application:

```
pkcs11-tool --module /path/to/your-library.so --list-slots
```

   This also prints the **Key ID** (`CKA_ID`) you need for the next step.
3. Enter the **Library path** and **Key ID** on the settings page.
4. Click **Test Token (no PIN)** to confirm the library loads and the key is found. Click **Test Token (with PIN)** (entering the PIN in the *PIN (test only)* field) to confirm the PIN itself is correct — this PIN is used only for the test and is never written to disk (Figure 5 shows a successful result).
5. On the main window, leave the **PIN** field empty if your token has a built-in PIN pad — the PIN is then requested on the device itself, and the PKCS#11 session is kept open so you are only asked once per session.

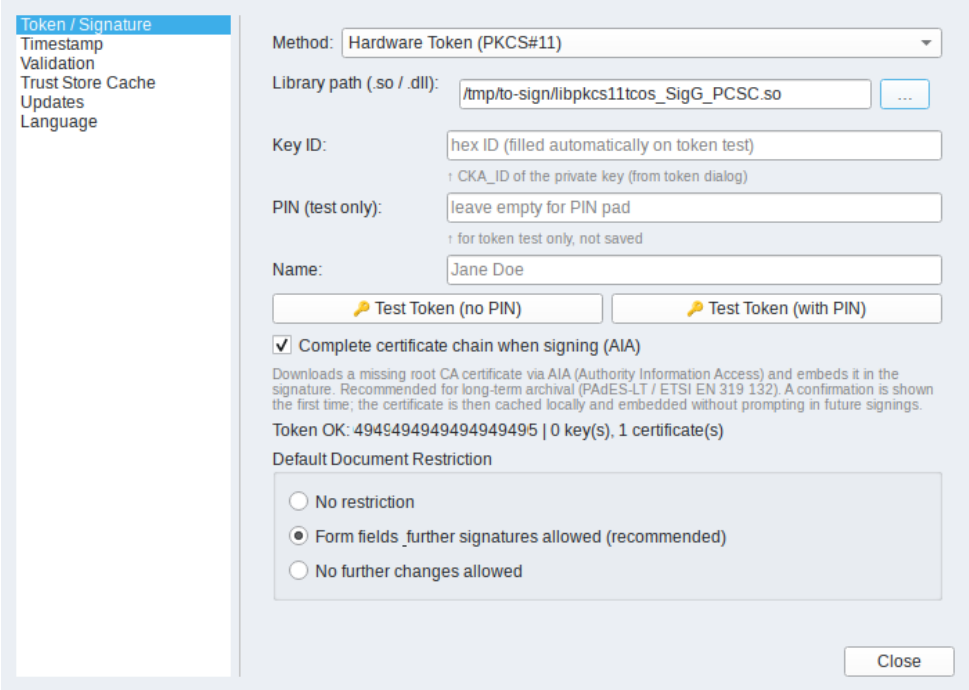

Figure 5. *Settings dialog in PKCS#11 mode with a successful "Test Token" result shown.*

---

6 Interface Overview

The main window is split into three areas:

- **Menu bar:** *File* (Open, Save, Save As, Quit), *Settings* (Settings…, Manage Profiles…), *Help* (About, License).
- **Toolbar:** Open file, Save fields, page navigation (previous/next page and an editable page-number field), a toggle between single-page and continuous scroll view, zoom controls (zoom out, an editable zoom percentage field, zoom in, fit-to-width, fit-to-height), a text-annotation tool toggle (chapter 9, keyboard shortcut `T`), and — on the right — **Sign** (`S`) and **Check signatures** (`C`).
- **PDF canvas** (left, largest area): displays the current page(s). Left-click and drag to draw a new signature field; click an existing field to select it; right-click to delete it. Existing PDF form fields (text fields, checkboxes, etc.) can also be filled in directly here — see chapter 8.
- **Right-hand panel:**
  - **Field list:** row 0 is always "Invisible signature" (sign without a visible field); below that, every signature field on the document, color-coded by category — free/unsigned fields you drew yourself, **locked** fields shown in orange (unsigned fields that already existed in a previously-signed PDF — they can be signed but not moved or deleted, because doing so would invalidate the existing signature), and **signed** fields shown in grey with a checkmark (read-only). All three colors are shown

together in Figure 6.

  - **PIN / Password field:** relabels itself depending on the active signing method (chapter 3) — the PIN for a hardware token, or the file password for a P12/PFX certificate.
  - **Enable Timestamp (TSA) checkbox:** turns on RFC 3161 timestamping for the next signature (see chapter 12).
  - **Appearance panel:** two tabs, *Text* and *Image/Layout*, controlling what the visible signature stamp looks like (see chapter 11).
- **Status bar:** shows the active profile name on the right; click it to quickly switch profiles.

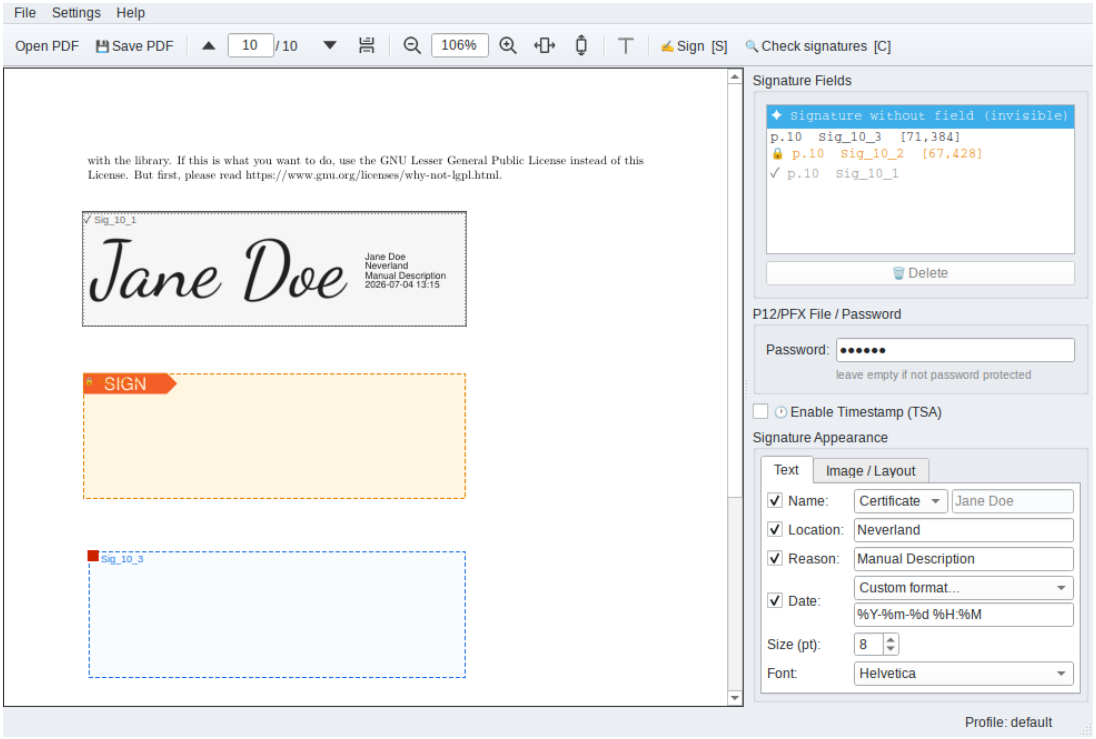

Figure 6. *Full main window with a PDF open, one signed field (grey), one locked field (orange), and one free field (blue) visible in the field list, so all three colors are shown at once.*

---

7 Converting LibreOffice "Sign_" Placeholder Fields

Some applications — LibreOffice in particular — cannot create real PDF signature fields at all. As a workaround, documents from such tools sometimes place ordinary **text form fields** whose name starts with `Sign_` (e.g. `Sign_1`, `Sign_Manager`) where a signature should later go.

7.1 Creating the placeholder fields in LibreOffice Writer

If you're authoring the document rather than just signing it, here's how to place a `Sign_*` placeholder in LibreOffice Writer. LibreOffice's form-control UI is not very discoverable — this is the minimal path, skipping everything not needed for a plain placeholder (database binding, list boxes, and so on).

1. Enable the form-control tools: **View → Toolbars → Form Controls** (older versions), or the **Form** menu directly (newer versions). Turn on **Design Mode** if it isn't already active.
2. Insert a **Text Box** control and draw it at the position where the signature should later appear (Figure 7).
3. Right-click the control. The context menu offers **two** similarly named entries — **Form...** and **Control...** — this is the single most common mistake here, since the Form's own Name property means something completely different (it names the invisible container the control lives in, not the control itself) and has

no effect on the exported PDF field name. Pick **Control...** (or select the control and press `F4`). On the **General** tab of that dialog, set **Name** to `Sign_` followed by something that identifies the signer, e.g. `Sign_Landlord` or `Sign_Tenant` — this exact name becomes the signature field's name in PDF QES Signer later (Figure 8).

4. Turn Design Mode back off, then **File → Export As → Export as PDF...**. On the **General** tab, under **Forms**, tick **Create PDF form** (Figure 9) — without this, LibreOffice flattens the control into plain page content instead of exporting it as a real, detectable form field.

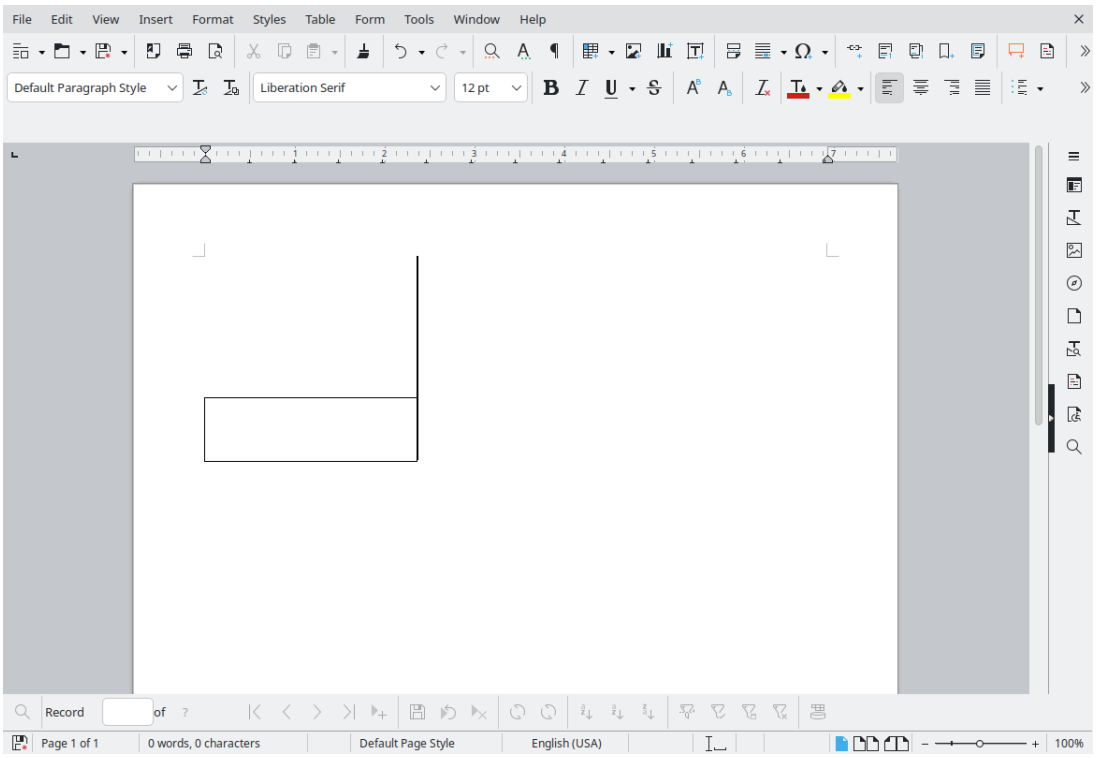

Figure 7. *A freshly drawn Text Box control on the page, in Design Mode.*

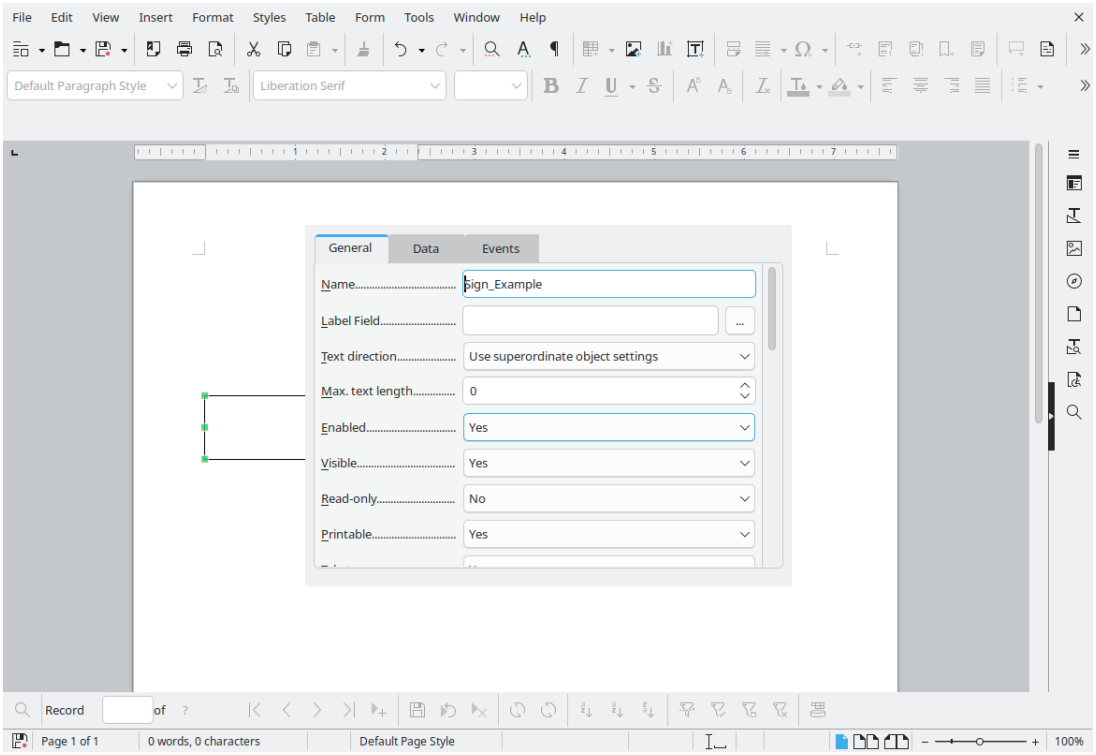

Figure 8. *Control Properties (not Form Properties!), General tab: the Name field is what PDF QES Signer reads back — it must start with* `Sign_`.

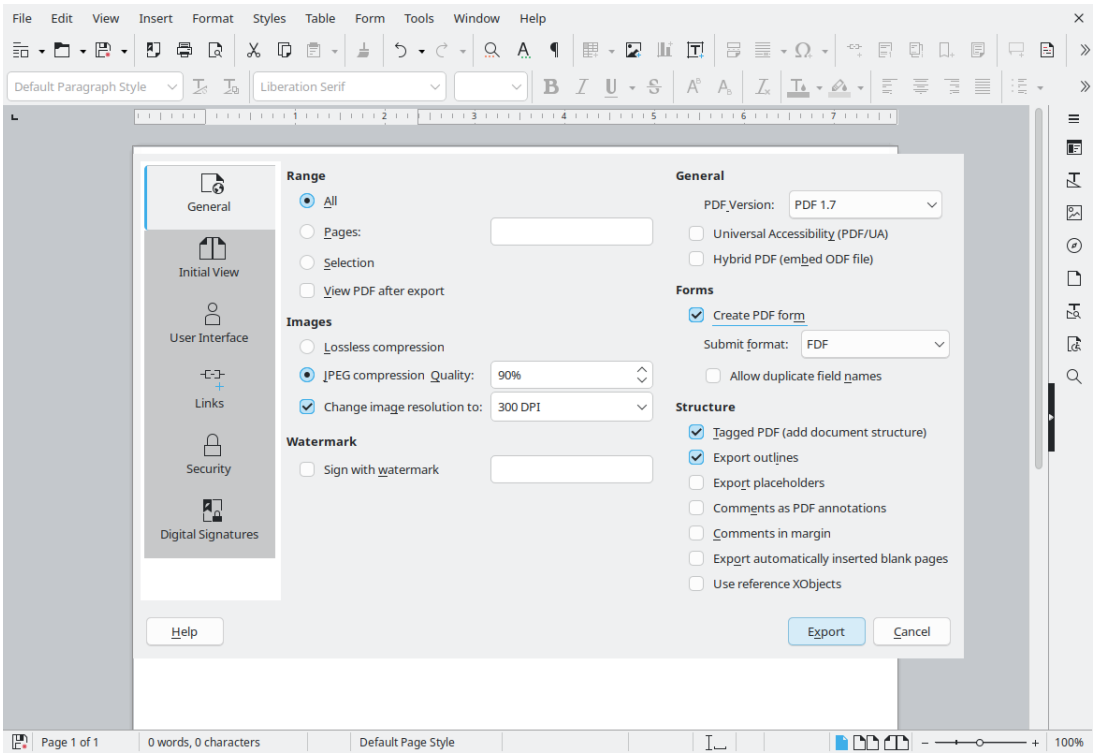

Figure 9. *PDF export dialog with the required state:* ***Create PDF form*** *(under Forms) ticked — without it, the placeholder never reaches the PDF as a real field.*

7.2 Opening the result in PDF QES Signer

When you open a PDF that contains such fields, PDF QES Signer detects them automatically and asks once whether to convert them into real signature fields. Confirming removes the text-field widgets and turns each one into a free, editable signature field (chapter 6) at the exact same position and page, ready to be signed like any field you drew yourself. Declining leaves the document unchanged — the fields remain plain, unusable text fields.

This detection can be turned off entirely on the **General** settings page (chapter 15) if you don't work with such documents, or if you'd rather convert them by hand each time.

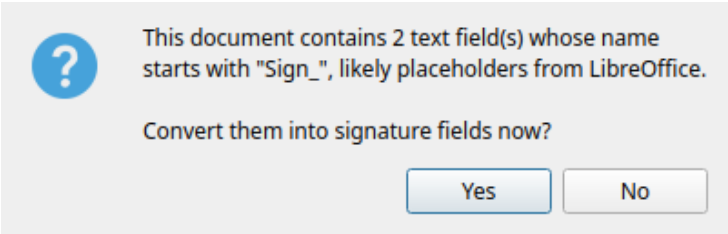

Figure 10. *The confirmation prompt shown when* `Sign_*` *placeholder fields are detected on open.*

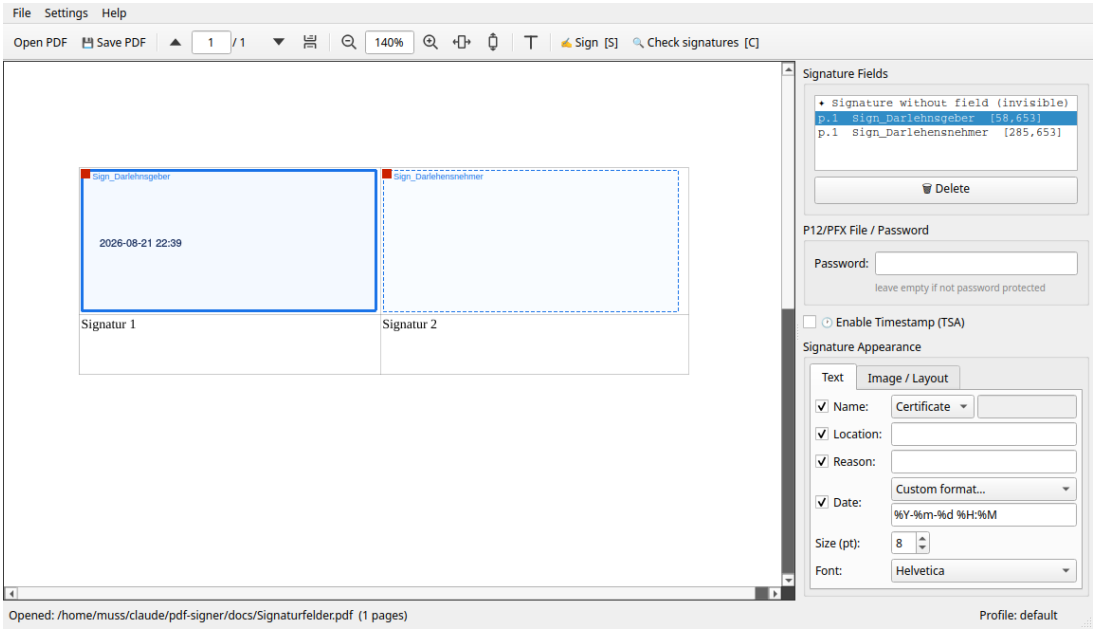

Figure 11. *After confirming: a real-world example (a loan agreement with two* `Sign_*` *placeholders, one per party) with both fields now converted to free signature fields at their original position, one selected to show its live appearance preview.*

---

8 Working with Form Fields

If the PDF you open already contains **form fields** (text fields, checkboxes, radio buttons, or comboboxes — created e.g. in Word, LibreOffice, or Acrobat), you can fill them in directly on the canvas, without any separate "edit mode": every field the application can edit is highlighted with a **green** tint and border as soon as the document is opened (Figure 12).

- **Text field:** click it to open an inline editor pre-filled with its current value (a multi-line box for multi-line fields); type your value and press `Enter` or click elsewhere to commit it. Press `Tab` / `Shift+Tab` to jump to the next/previous text field, in reading order.

- **Checkbox:** a single click toggles it on/off; a green checkmark is drawn when checked.
- **Radio button:** clicking one option in a group selects it and automatically clears the other options in the same group.
- **Combobox:** click it to open a dropdown of the choices defined in the PDF; selecting one commits it immediately.

Text is displayed using the font and size the PDF itself specifies for that field; if the field doesn't specify a size, it is sized automatically to fit the field's height. If a multi-line text field's content no longer fits after you finish typing, the font is shrunk automatically so everything remains visible.

Some PDF form field types (in particular **list boxes**) are not supported for in-app editing. If a document contains any, a status-bar notice appears for a few seconds: *"This document contains fields that cannot be edited here. Please verify their content before signing."* Check such fields in another PDF viewer before signing.

To save the values you entered, use **File → Save with fields (copy)...** or the **Save PDF** toolbar button. This writes your entries into a new PDF file — the form fields remain live/interactive for other PDF viewers; this step does **not** flatten the form (values are only permanently fixed in place, alongside the visible appearance, once you actually sign — see chapter 10).

> **Important:** form fields can only be edited while the document has no signature fields yet. As soon as *any* signature field exists on the document — whether already signed, or merely present-but-unsigned (`locked`, see chapter 6) — all form fields become read-only, even if the document's signing permission (docMDP) would technically still allow form fill-in. Fill in all form fields *before* placing the first signature.

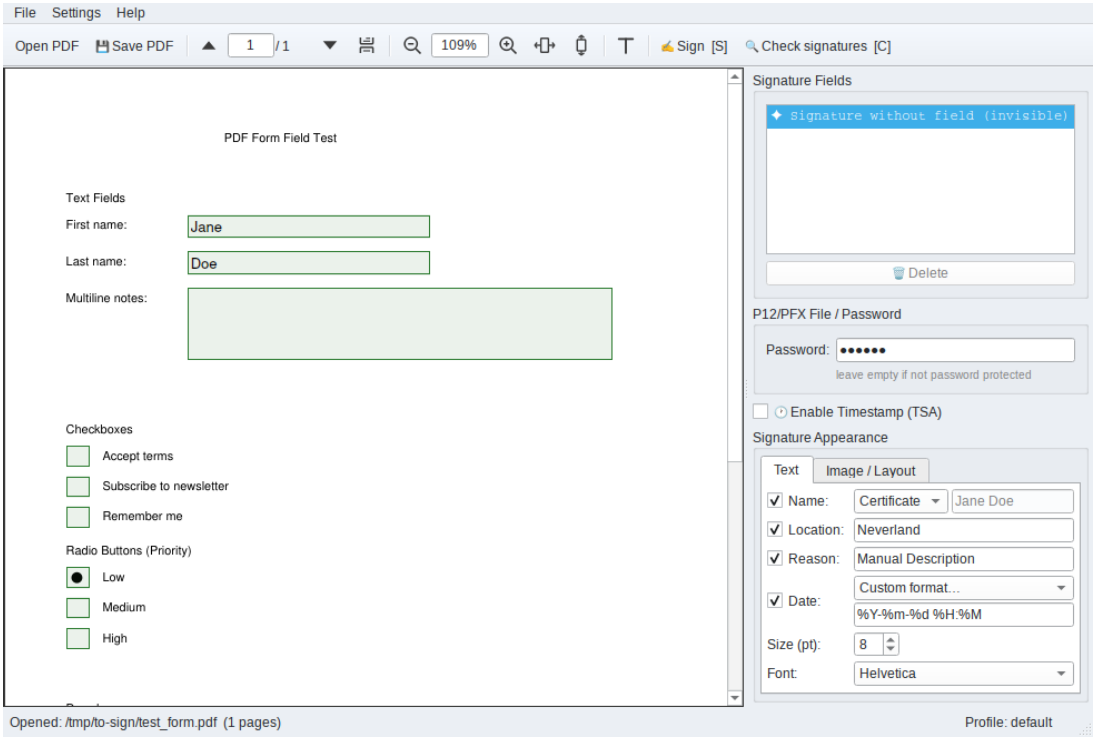

Figure 12. *A PDF with editable form fields: text fields, checkboxes, radio buttons, a dropdown, and a multi-select list box.*

9 Adding Text Annotations

Independently of any form field in the PDF, you can place free-form text anywhere on the page yourself — a date, a note, initials next to where a signature will go, and so on.

Click the **text-annotation tool** toggle in the toolbar (or press `T`) to enter text mode. A second toolbar appears above the canvas with:

Control	Meaning
Font	One of 12 standard PDF fonts: Helvetica, Times, and Courier, each in Regular, Bold, Oblique, and Bold Oblique. No font embedding is required, so text always renders identically everywhere.
Size (`A↕`)	6–72 pt.
Character spacing (`A↔`)	0–50 pt of extra space inserted between letters.
Color	Click the swatch button to pick a text color.

While in text mode, **click anywhere on the page** to place a new text box and start typing immediately; the current toolbar settings apply to it. Press `Tab` / `Shift+Tab` to jump between text boxes in the order they were placed, same as for form-field text boxes (chapter 8). Drag the small red handle in a box's top-left corner to reposition it. Right-click a box to delete it immediately (no confirmation, unlike deleting a signature field). A box you leave completely empty is discarded automatically once you click elsewhere.

Press `Escape`, or click the toolbar toggle again, to leave text mode — boxes with content stay on the page, they just lose the editing cursor until you re-enter text mode and click them again.

Like form fields (chapter 8), text annotations can only be added while the document has **no signature fields yet**; the toolbar toggle is disabled once any signature field exists. At the moment you sign, text annotations are permanently burned into the page content, following the same "flatten before finalizing" approach used for form fields — but unlike form fields, the text stays **selectable**, not a picture of text: the font is embedded as a subset so it renders identically everywhere it's viewed. Any *foreign* free-text annotation already present in the PDF from another application (e.g. a comment added in Acrobat) is burned in too, but as a rasterized image instead, since the application does not know that annotation's original font.

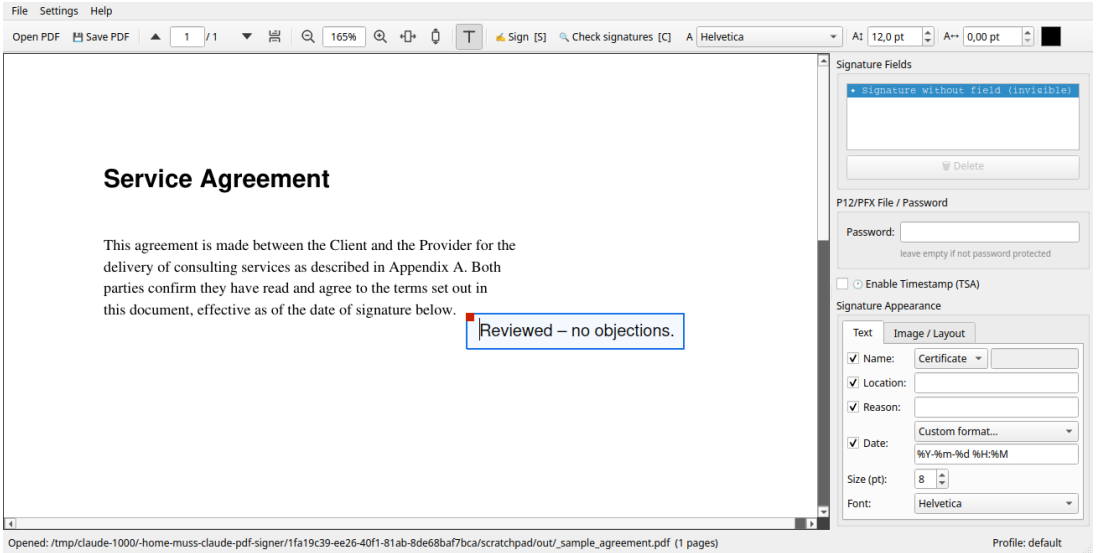

Figure 13. *Text mode active: the formatting toolbar (font, size, character spacing, color) and one placed text annotation, selected and ready to edit.*

10 Signing a PDF

1. **Open a PDF** (*File* → *Open*, or pass it as a command-line argument).
2. **Place a signature field:** left-click and drag on the page to draw a rectangle where the visible signature stamp should appear. Alternatively, select row 0, *Invisible signature*, in the field list to sign without any visible mark.
3. **Configure the appearance** (chapter 11) — changes are previewed live on the canvas.
4. **Select the field** you want to sign in the field list (if not already selected).
5. Optionally check **Enable Timestamp (TSA)** to embed an RFC 3161 timestamp (chapter 12).
6. If signing with a hardware token, enter the **PIN** (or leave it empty for a hardware PIN pad).
7. Click **Sign** in the toolbar.
8. Choose where to save the signed PDF (a filename with a language-specific "-signed" suffix is suggested by default).
9. **First signature only:** a dialog (Figure 14) asks which **document restriction** (docMDP) to apply — *No restriction*, *Form fields & further signatures allowed* (recommended), or *No further changes allowed*. This choice is permanent for the document and is remembered as the default for next time.
10. If signing with a PFX file and the password field was left empty or was wrong, you are prompted again until it is correct or you cancel.
11. Signing runs in the background; once complete, the signed PDF is **reloaded automatically** so you can place and apply further signatures in sequence (e.g. multiple signers on the same document).

Existing unsigned fields found in an already-signed PDF are shown as **locked** (orange) rather than free (blue): they can still be signed, but not moved or deleted, since doing so would invalidate the existing signature's cryptographic hash over the file — see Figure 6 above for what a drawn field and its live appearance preview look like alongside the field list.

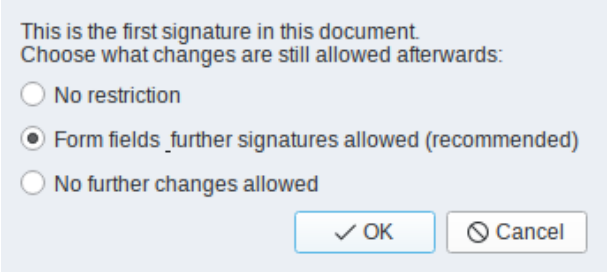

Figure 14. *The docMDP restriction dialog shown on first signing.*

11 Configuring the Signature Appearance

The Appearance panel on the right side of the main window has two tabs (Figure 15 and Figure 16).

11.1 Text tab

Field	Meaning
Name	Checkbox to show the signer's name; a dropdown selects the source — *from certificate* (the CN) or *custom* (free text you type in).
Location	Free-text signing location (e.g. "Berlin"), stored in the PDF signature metadata.

Field	Meaning
Reason	Free-text signing reason (e.g. “Approved”), stored in the PDF signature metadata.
Date	Checkbox plus a dropdown of common date/time formats with a live example, or *Custom…* to type your own `strftime`-style format. The date shown is replaced by pyhanko with the actual cryptographic timestamp at signing time.
Font size	5–24 pt.
Font family	One of the 14 standard PDF fonts (no font embedding required, so the signature always renders identically everywhere).

11.2 Image/Layout tab

Field	Meaning
Image	Optional PNG (transparency supported) shown inside the signature stamp, e.g. a scanned handwritten signature or a logo. Browse or Clear.
Layout	Whether the image appears to the left or right of the text.
Border	Optional thin border around the entire signature field.
Image/text ratio	Slider (10–70%) controlling how much of the field width the image occupies versus the text.

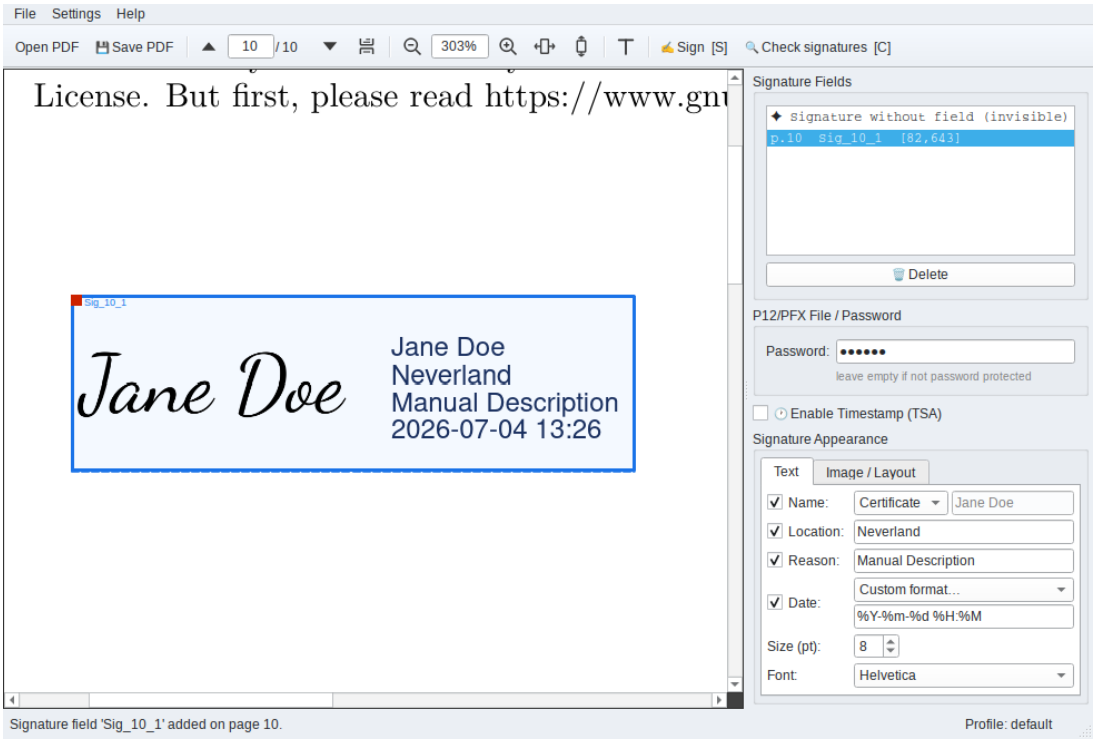

Figure 15. *Appearance dialog, Text tab.*

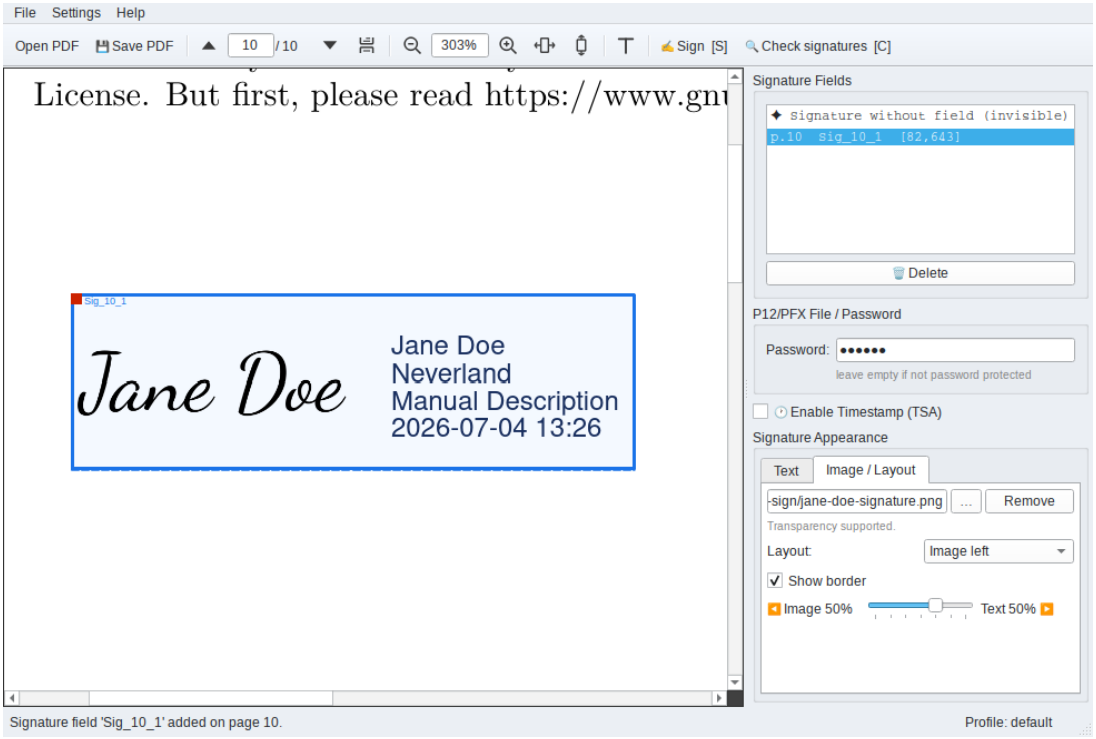

Figure 16. *Appearance dialog, Image/Layout tab, with an image loaded so the ratio slider's effect is visible in the canvas preview.*

---

12 Configuring the Timestamp (TSA)

A plain signature only proves the document existed, unmodified, at the moment it was signed *according to your own computer's clock* — which nobody else can independently verify. An RFC 3161 **timestamp** fixes this: an independent Time Stamp Authority (TSA) cryptographically certifies the exact signing time using its own trusted clock, instead of yours. This matters most for long-term validity — if your certificate later expires or is revoked, a trustworthy timestamp still proves the signature was created while the certificate was still valid. This is what the PAdES **T** level shown in chapter 13 refers to.

Open **Settings → Settings... → Timestamp** (Figure 17).

- **TSA URL:** the address of an RFC 3161 timestamp authority. Leave empty to use the built-in default (`http://tss.accv.es:8318/tsa`). Any RFC 3161-compliant TSA can be used instead.
- **Embed revocation status for long-term archival (OCSP/PAdES-LTA):** when enabled, an OCSP revocation check is embedded alongside the timestamp, producing a **PAdES-LTA** signature suitable for long-term archival. This requires a working TSA and a CA-issued certificate with an OCSP service (not available for self-signed certificates, see chapter 4). Trust roots are taken from the Mozilla CA bundle (certifi) plus any CA certificates present on the token — no system CA store is required. If the OCSP fetch fails, signing still proceeds normally with a warning.

The **Enable Timestamp (TSA)** checkbox on the main window turns timestamping on or off for the *next* signature; the settings page above controls *which* TSA is used and whether LTA information is embedded when it is on.

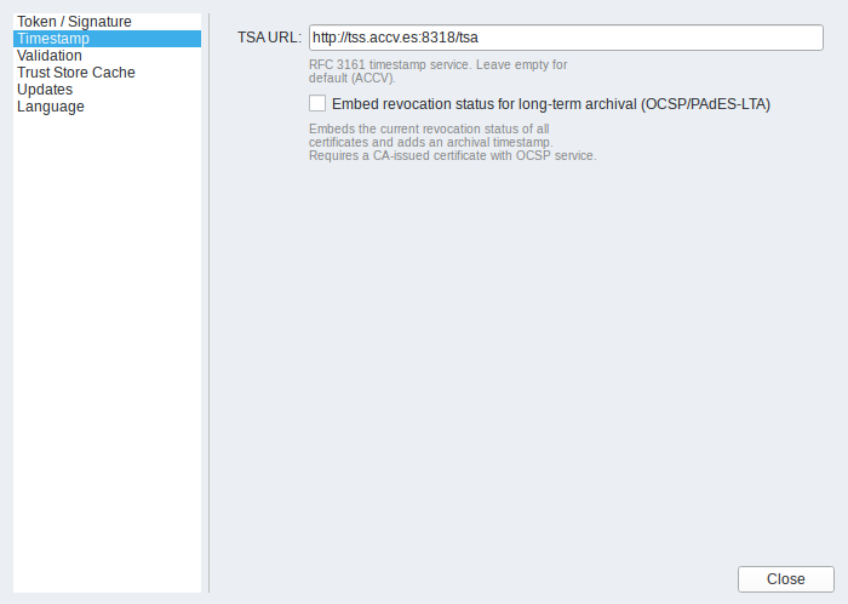

Figure 17. *Settings dialog, Timestamp page.*

13 Checking Signatures

A signature being *present* on a PDF is not the same as it being *valid*. Checking a signature means verifying three separate things: that the document's bytes haven't changed since it was signed (integrity), that the signing certificate's chain of trust actually leads to a Certificate Authority someone chooses to trust, rather than just to itself (authenticity — see chapter 17), and that the certificate wasn't revoked at the time of signing. This chapter's revision tree shows exactly that information for every signature layered onto the document over time — "revision" is explained in chapter 17 as well.

Open **Sign → Check signatures**. This opens a non-modal window (the main window stays interactive) showing every revision of the PDF, newest first (Figure 18).

For each **signed** revision:

- Signer name, signing time (the signer's own claim, or the TSA-confirmed time if a timestamp is present), cryptographic integrity status, and the PAdES conformance level — **B** (basic), **T** (timestamped), **LT** (long-term), or **LTA** (long-term with archive timestamp) — with a plain-language explanation of what each level means.
- A **certificate chain status** row, color-coded: valid, self-signed, unknown root, revocation unknown, incomplete, expired, or revoked, with a tooltip explaining the status. Click **Details →** to open a floating window (Figure 19) listing every certificate in the chain with its role, validity period, source, and OCSP status. This also applies separately to the embedded TSA timestamp's own certificate chain.
- When the *automatic fetch* validation mode is active (Settings → Validation), chains are additionally checked against the EU List of Trusted Lists and the relevant national Trust Service List, based on the certificate's country.

Other useful controls:

- **Show all revisions** reveals unsigned revisions too (form field fills, DSS updates, metadata) — automatically enabled if a modification warning is active.
- Clicking any revision switches the PDF viewer to show the document exactly as it looked at that point in time.
- If the document was modified after its last signature in a way that affects visible content, a warning banner appears immediately on opening — you don't need to open this dialog to notice.
- If the document carries a `docMDP P=1` certification (no changes allowed), a warning banner appears and all editing/signing is disabled.

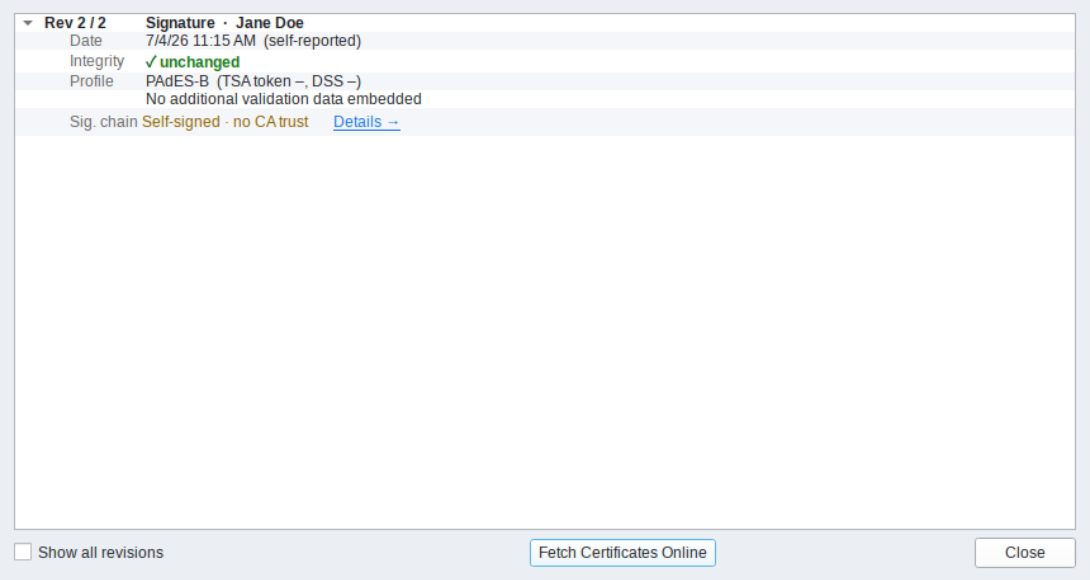

Figure 18. *Revision tree with a chain-status label.*

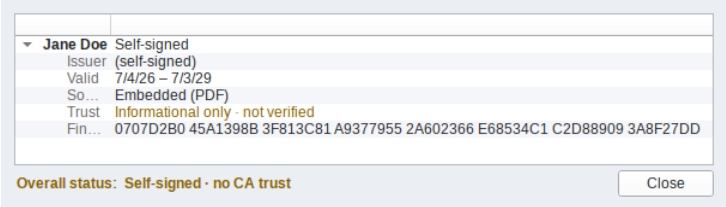

Figure 19. *Certificate chain inspector ("Details →") open.*

14 Managing Profiles

A **profile** bundles a complete signing configuration — signing method, TSA URL, appearance defaults, docMDP default — under a name, so you can quickly switch between identities (e.g. signing for yourself vs. for an organization) without re-entering everything.

- **Settings → Manage Profiles...** opens a list of all profiles (Figure 20). The active one is marked. Buttons: **New**, **Rename**, **Delete**, **Close**. Double-clicking a profile activates it and closes the dialog.
- The **profile name in the status bar** (bottom-right of the main window) is also clickable, for a quick switch without opening the full manager.

Profiles are stored as `~/.config/pdf-signer/profiles/<name>.ini`; a separate `settings.ini` holds settings that are not profile-specific (language, update channel).

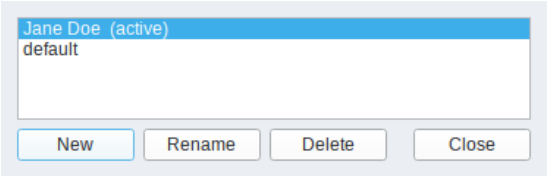

Figure 20. *Profile manager dialog with two or more profiles listed.*

15 Additional Settings

Validation page:

- **Auto-fetch revocation status during validation:** fetches OCSP status online while checking signatures. Requires network access to the CA's OCSP service — consider the data-privacy implications before enabling this for documents from third parties.

Trust Store Cache page (Figure 22):

- Shows the status of the locally cached LOTL URL list and national TSL files (validity date, size) and provides buttons to clear the TSL cache, the AIA-fetched certificate cache, or both. Cached data is stored under `~/.config/pdf-signer/tsl_cache/` and `~/.config/pdf-signer/aia_cert_cache/`.

Updates page:

- **Automatically check for updates on startup.**
- **Update Channel:** *Stable* (recommended) or *Develop* (pre-releases). Switching to Develop shows a confirmation warning first.

General page:

- **Language:** choose the UI language (German, English, French, Spanish, Italian, Dutch, Polish, or Portuguese). Takes effect immediately, no restart needed.
- **Automatically convert Sign__ placeholder fields into signature fields on open:** enabled by default; controls the LibreOffice-workaround detection described in chapter 7. Turn it off if you never work with such documents, or prefer to decide by hand every time instead of being asked on open.

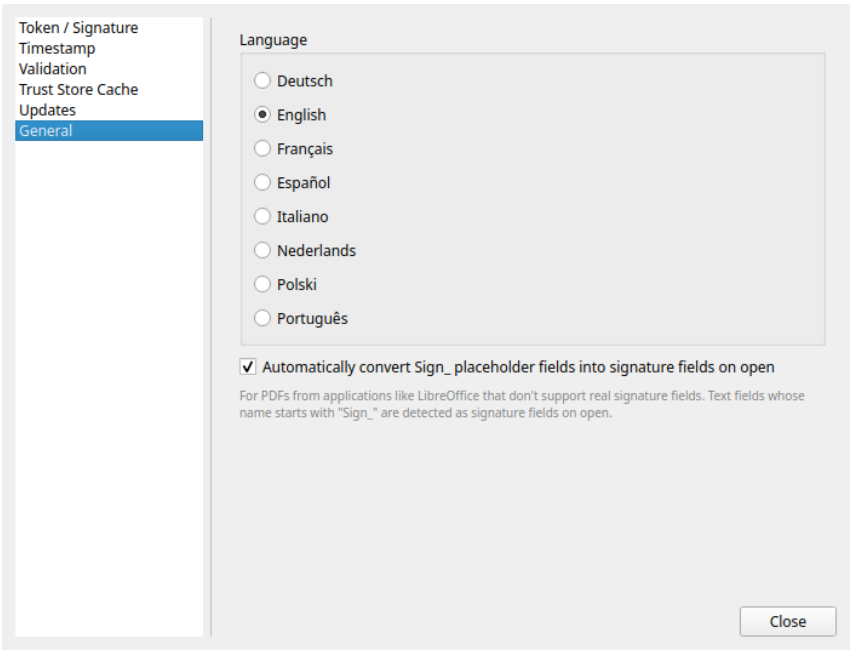

Figure 21. *Settings dialog, General page.*

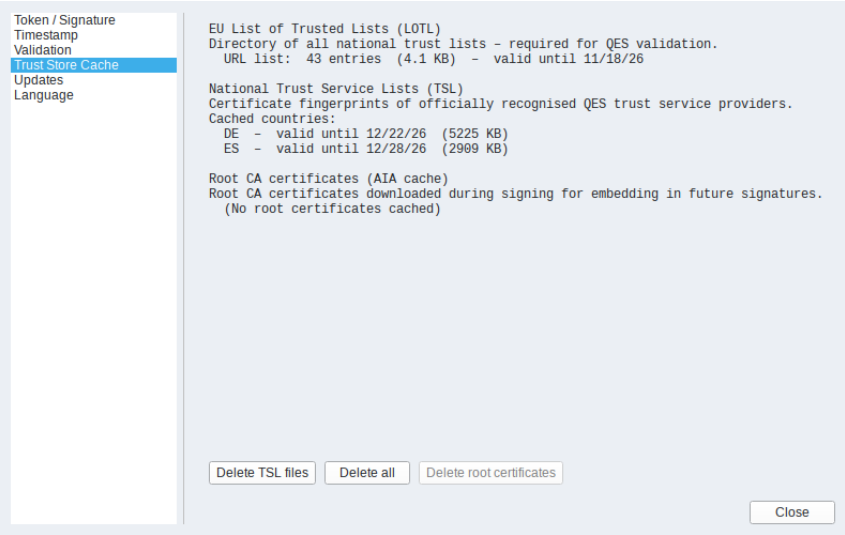

Figure 22. *Trust Store Cache page showing populated cache info.*

---

16 Troubleshooting

Crash log: unexpected errors are written to `~/.local/share/pdf-signer/crash-YYYY-MM-DD.log` (kept for 7 days). If the application shows a crash dialog, you can usually choose to continue or quit; either way, check this log file for details before reporting an issue.

Common issues:

Symptom	Likely cause / fix
Token not found / Key ID unknown	Run `pkcs11-tool --module <lib> --list-slots` to confirm the card is recognized and to read the correct Key ID.
"Wrong password" when signing with a PFX file	The password you entered does not unlock the private key; you are prompted again — check for typos or a forgotten password change.
Timestamp / TSA request fails	The configured TSA server may be temporarily or permanently unreachable; try the built-in default or another public RFC 3161 TSA.
Cannot edit or sign a document at all	The document likely carries a `docMDP P=1` certification signature (no changes allowed) — this is enforced by design and cannot be bypassed.
PAdES-LTA option greyed out / OCSP fails	Self-signed certificates have no OCSP responder; use a CA-issued certificate for long-term archival.

If none of the above resolves the problem, please report an issue on the project's Codeberg repository (the GitHub mirror is read-only and does not accept issues), including the relevant crash log if one was created.

17 Appendix: How PDF Signing Actually Works

The previous chapters refer to a few concepts — revisions, certificates, certificate chains — without explaining them. This appendix gives the short version.

17.1 Revisions and incremental updates

A PDF file can be extended by *appending* new data to its end without touching any of the bytes already there — this is how PDF viewers add comments, fill in form fields, or add signatures without needing to rewrite the whole file. Each such appended chunk is called a **revision**.

When you sign a PDF, PDF QES Signer does not rewrite the file: it appends a new revision containing a signature object and a `ByteRange` — the exact list of byte offsets the signature covers (everything up to that point, skipping only the signature's own placeholder). The private key signs a cryptographic hash of those bytes. This has two direct consequences you will encounter elsewhere in this manual:

- **Multiple signatures can coexist.** A second signature simply starts a new revision covering everything before it, including the first signature. This is how several people can sign the same document in sequence (chapter 10).
- **Later changes are detectable.** If any byte inside an earlier signature's `ByteRange` no longer matches what was originally hashed, that signature is reported as broken/modified — this is exactly the warning banner mentioned in chapter 13. The docMDP permission level (chapter 10) controls what kinds of later revisions are still allowed at all.

17.2 Certificates and key pairs

A signature relies on **public-key cryptography**: a key pair consists of a private key (kept secret, used to sign) and a mathematically related public key (shared openly, used to verify). Signing hashes the document and encrypts that hash with the private key; verifying decrypts it with the public key and checks the hash still matches — proving the document is unchanged and that whoever holds the private key signed it.

A **certificate** is what makes the public key meaningful: it bundles the public key together with an identity (name, organization, email, …) and is itself digitally signed — either by a Certificate Authority (CA) that checked that identity beforehand, or, for a **self-signed** certificate (chapter 4), by the same key pair it describes (i.e., you vouching for yourself).

17.3 Certificate chains and trust

A certificate is validated by following its issuer up to a **root** certificate that is inherently trusted (a "trust anchor"):

```
End-entity certificate  (yours — the signer)
        ↑ issued by
Intermediate CA certificate  (0 or more of these)
        ↑ issued by
Root CA certificate  (the trust anchor)
```

This application trusts the same root CAs as most browsers (the Mozilla CA bundle, via `certifi`), plus — where the automatic validation mode is enabled — the EU's national Trust Service Lists, for certificates that claim to be qualified under eIDAS (chapter 13). A **self-signed** certificate is its own root, so it is never found in either list — it always shows as "unknown root" / untrusted unless the person verifying it has manually decided to trust that specific certificate.

Finally, a CA can **revoke** a certificate before its expiry date (e.g., after a key compromise). Checking whether that happened requires contacting the CA online (OCSP) — this is what the *auto-fetch revocation status* and *PAdES-LTA* options referenced in chapter 12 and chapter 15 are for.